

MiniANN MVP: Master Technical Report

An Object-Oriented Feedforward Neural Network Engine & Workbench in C++20

MiniANN Engineering Team | Final Project Capstone Submission

Repository Branch: p2 | Standard: ISO/IEC C++20 | Zero ML Dependencies

Abstract

MiniANN MVP is a complete, production-grade Artificial Neural Network (ANN) simulation framework and interactive laboratory application written from scratch in modern C++20. Designed under the strict pedagogical constraint of **zero external machine learning or linear algebra libraries** (e.g., no PyTorch, TensorFlow, Eigen, or BLAS), the system implements complete forward propagation, analytic backpropagation, gradient verification, multiple optimization strategies, regularization mechanics, and dataset preprocessing natively. Built alongside the core engine is an interactive desktop workbench implemented using Qt6, featuring asynchronous worker threading, live parameter streaming, real-time loss/accuracy charting, 2D decision boundary visualization, and feedforward network state inspection. This report serves as the definitive reference for the project, detailing system architecture, object-oriented design patterns, mathematical formulations, end-to-end execution traces, automated test suites, and UI/UX engineering.

Contents

1	Executive Summary & Problem Statement	2
2	End-to-End System Architecture	2
2.1	Architectural Pipeline Flow	2
2.2	Pipeline Execution Stages	2
3	Object-Oriented Programming (OOP) Implementation	3
3.1	Core OOP Pillars in MiniANN	3
3.2	Design Patterns Matrix	5
4	UML Class Diagrams	5
4.1	Core Domain Hierarchy (Model & Activation)	5
4.2	Optimization, Loss & Visitor Architecture	5
5	Mathematical Formulations & Algorithmic Logic	5
5.1	Forward Propagation	5
5.2	Analytic Backpropagation Equations	5
5.3	Regularization & Gradient Norm Clipping	5
5.4	Adaptive Optimization Algorithms	6
6	End-to-End Dry Run Trace (Concrete Walkthrough)	6
6.1	Initial State Setup	6
6.2	Forward Pass Execution	6
6.3	Backward Pass Execution	7
6.4	Parameter Update (RMSprop Step)	7

7	Interactive Workbench (GUI/UX) Architecture	8
7.1	Asynchronous Threading Model (LiveBridge)	8
7.2	Visualization Canvas Modules	8
7.3	UI Polish & Workflow Enhancements	8
8	Automated Verification & Test Suite	8
9	Conclusion & Future Roadmap	8
9.1	Architectural Readiness for 3D Visualization	9

1 Executive Summary & Problem Statement

Modern machine learning education and engineering often suffer from high levels of abstraction. While libraries like PyTorch or TensorFlow allow rapid prototyping, they obscure the low-level computational mechanics, numerical stability challenges, memory layout trade-offs, and software design patterns that underpin artificial neural networks.

The **MiniANN MVP** project addresses this by developing an end-to-end neural network engine and visualization workbench adhering to the following core tenets:

- **Complete First-Principles Implementation:** Every component—from neuron dot-products and analytic activation derivatives to backpropagation chain rule computations and momentum buffers—is implemented natively in C++ standard library structures (`std::vector<double>`).
- **Exemplary Object-Oriented Design:** The architecture rigorously adheres to the SOLID principles, leveraging nine foundational GoF design patterns (Factory, Strategy, Observer, Visitor, Composite, Chain of Responsibility, Façade, Bridge, Memento) to guarantee modularity, maintainability, and loose coupling.
- **Verifiable Mathematical Correctness:** Analytic backpropagation gradients are verified against centered finite-difference approximations via an automated gradient checker ensuring relative errors remain below 10^{-7} .
- **Real-Time Interactive Telemetry:** A zero-latency, multi-threaded GUI provides dynamic real-time visualization of loss curves, accuracy trajectories, confusion matrices, 2D decision boundaries, and node weight magnitudes without blocking training execution.

2 End-to-End System Architecture

2.1 Architectural Pipeline Flow

The application follows a strictly layered, unidirectional pipeline architecture. The separation between data ingestion, model representation, training execution, and visualization ensures that the computational engine remains completely decoupled from the presentation layer.

2.2 Pipeline Execution Stages

1. **Ingestion Stage:** Raw data is read from disk or built-in synthetics via `IDataSource`. A Chain of Responsibility resolves label column ambiguities.
2. **Validation & Normalization Stage:** The dataset is validated using `CompositeValidator` (checking for NaNs, dimension mismatches, class degeneracies) and normalized using an `INormalizer` strategy (Z-Score or MinMax).
3. **Construction Stage:** The network topology is assembled via `NeuralNetwork`, composing `Layer` containers populated by `Neuron` units initialized with Xavier/He weight distributions.

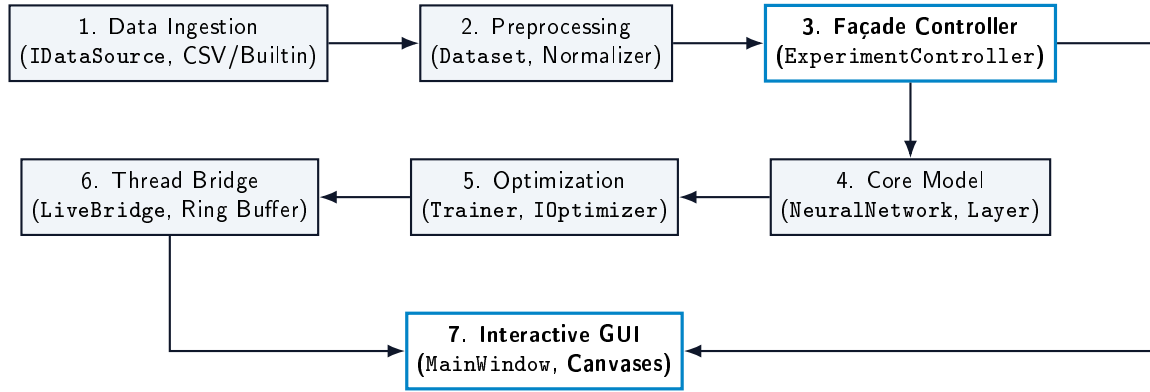


Figure 1: MiniANN End-to-End System Pipeline and Component Interaction

4. **Training Stage:** **Trainer** executes forward and backward passes over mini-batches. Optimizers (SGD, Momentum, Adam, RMSprop) update weights, modulated by learning rate schedulers, L_2 weight decay, and gradient clipping.
5. **Observer Telemetry Stage:** During each epoch, an **ICallback** chain logs metrics, enforces early stopping, and posts atomic snapshots to **LiveBridge**.
6. **Rendering & Export Stage:** The Qt GUI worker consumes snapshots asynchronously, re-drawing canvas widgets, while **Serializer** and report exporters generate standalone persistence artifacts.

3 Object-Oriented Programming (OOP) Implementation

The MiniANN codebase was engineered as a benchmark of modern object-oriented software design. Below is a comprehensive breakdown of how OOP fundamentals and design patterns are utilized across the architecture.

3.1 Core OOP Pillars in MiniANN

- **Encapsulation:** All mathematical states (weights, biases, cached net inputs z , outputs a , and accumulated gradients) are declared **private** within **Neuron** and **Layer**. Access is granted solely through well-defined accessor and mutator methods, preserving internal invariants.
- **Abstraction:** High-level training routines depend strictly on abstract interface contracts (**IActivation**, **IOptimizer**, **ILoss**, **INormalizer**, **ICallback**). The training engine operates without knowledge of concrete mathematical formulas.
- **Polymorphism:** Dynamic dispatch is executed through pure virtual interfaces. Activation computations, optimization steps, and callback hooks are resolved at runtime without monolithic conditional branching.
- **Inheritance & Interface Segregation:** MiniANN enforces shallow inheritance trees with virtual destructors. Interface contracts are focused and minimal, ensuring clients are not forced to implement unused methods.

Pattern	Component / Class	Architectural Justification & Benefit
Factory	ActivationFactory, OptimizerFactory, LossFactory, NormalizerFactory	Decouples object instantiation from business logic. Allows string-based configuration from GUI without leaking concrete types.
Strategy	IOptimizer (SGD, Adam, RMSprop), INormalizer, ILRScheduler	Encapsulates interchangeable algorithms within standard interfaces. Enables swapping optimization or normalization mechanics at runtime.
Observer	ICallback, EarlyStoppingCallback, LiveBridgeCallback	Implements decoupled event listening. Training engine broadcasts epoch metrics without coupling to console logs or GUI widgets.
Visitor	INetworkVisitor, WeightDecayVisitor, NeuronInspectorVisitor	Separates operations from object structures. Allows inspecting or modifying network hierarchies without polluting Neuron classes.
Composite	CompositeValidator, CallbackList	Treats individual objects and compositions uniformly. Aggregates multiple dataset sanity checks or training callbacks into single execution nodes.
Chain of Resp.	TargetSelectorChain	Decouples request sender from receivers. Successively analyzes dataset column headers and value distributions to auto-detect classification labels.
Façade	ExperimentController	Provides a high-level, unified interface over ingestion, validation, initialization, training, and evaluation subsystems.
Bridge	LiveBridge	Decouples background thread producer execution from Qt GUI consumer rendering via mutex-guarded ring buffers and atomics.
Memento	Serializer	Captures and restores network state vectors (weights, biases, layer configurations) without violating object encapsulation.

Table 1: Summary of GoF Design Patterns Implemented in MiniANN MVP

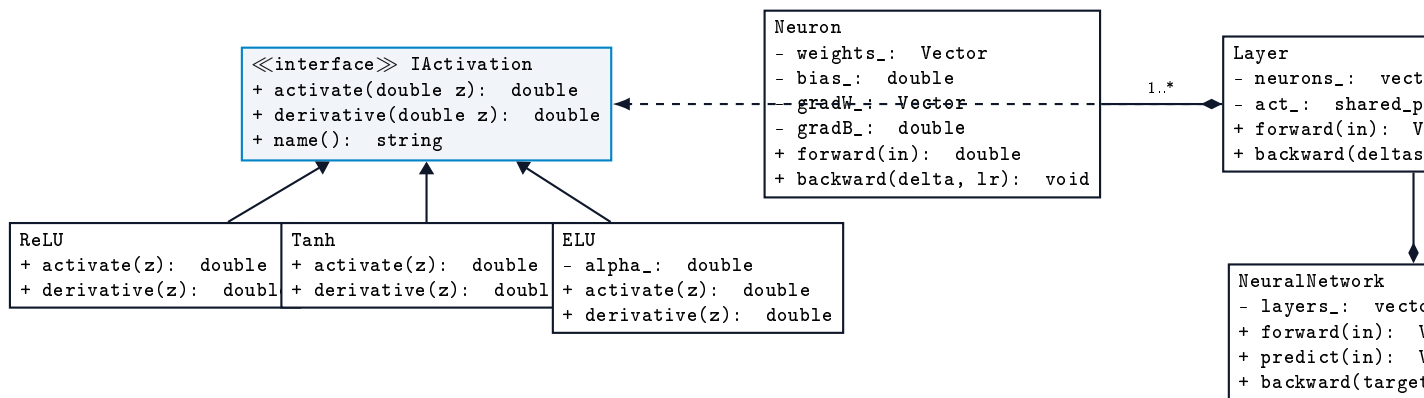


Figure 2: UML Class Diagram: Core Neural Network Model and Activation Hierarchy

3.2 Design Patterns Matrix

4 UML Class Diagrams

4.1 Core Domain Hierarchy (Model & Activation)

4.2 Optimization, Loss & Visitor Architecture

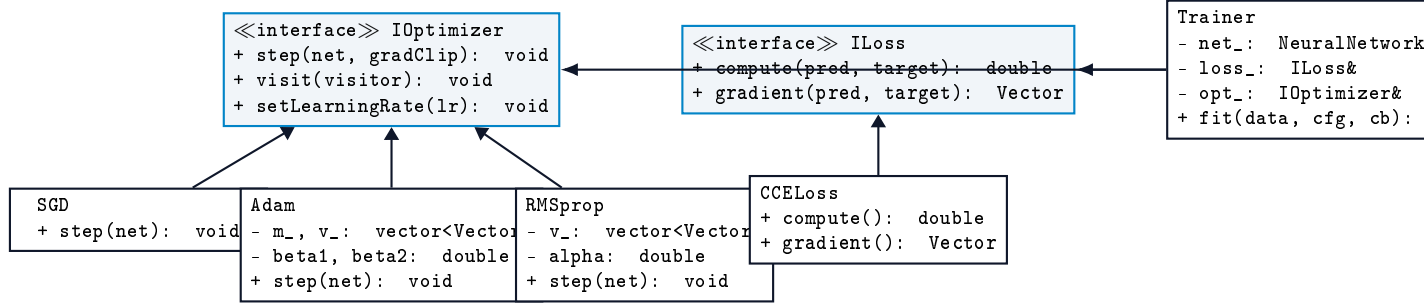


Figure 3: UML Class Diagram: Optimization and Loss Abstractions

5 Mathematical Formulations & Algorithmic Logic

5.1 Forward Propagation

For an input vector $\mathbf{x} = \mathbf{a}^{[0]} \in \mathbb{R}^{d_{\text{in}}}$, the activations at layer $l \in \{1, \dots, L\}$ with n_l neurons are evaluated as:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]} \quad (1)$$

$$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]}) \quad (2)$$

where $\mathbf{W}^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}}$ represents the layer weight matrix, $\mathbf{b}^{[l]} \in \mathbb{R}^{n_l}$ the bias vector, and $f^{[l]}$ the element-wise activation function.

5.2 Analytic Backpropagation Equations

Given target $\mathbf{y}$ and loss $\mathcal{L}(\mathbf{a}^{[L]}, \mathbf{y})$, the output layer error delta vector $\boldsymbol{\delta}^{[L]}$ is defined as:

$$\boldsymbol{\delta}^{[L]} = \nabla_{\mathbf{a}^{[L]}} \mathcal{L} \odot f^{[L]'}(\mathbf{z}^{[L]}) \quad (3)$$

For hidden layers $l = L-1, L-2, \dots, 1$, error signals propagate backward via:

$$\boldsymbol{\delta}^{[l]} = \left(\mathbf{W}^{[l+1]T} \boldsymbol{\delta}^{[l+1]} \right) \odot f^{[l]'}(\mathbf{z}^{[l]}) \quad (4)$$

The accumulated gradients with respect to parameters over a mini-batch of size B are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[l]}} = \frac{1}{B} \sum_{i=1}^B \boldsymbol{\delta}_i^{[l]} \left(\mathbf{a}_i^{[l-1]} \right)^T, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[l]}} = \frac{1}{B} \sum_{i=1}^B \boldsymbol{\delta}_i^{[l]} \quad (5)$$

5.3 Regularization & Gradient Norm Clipping

- **L_2 Regularization (Weight Decay):** With penalty coefficient λ , the weight gradients receive an additive penalty:

$$\mathbf{g}_W \leftarrow \mathbf{g}_W + \lambda \mathbf{W} \quad (6)$$

- **Global Gradient Norm Clipping:** To prevent exploding gradients, the total Frobenius norm over all parameters is computed:

$$\|g\|_2 = \sqrt{\sum_l \|\mathbf{g}_{W^{[l]}}\|_F^2 + \sum_l \|\mathbf{g}_{b^{[l]}}\|_2^2} \quad (7)$$

If $\|g\|_2 > g_{\max}$, all gradients are rescaled: $\mathbf{g} \leftarrow \mathbf{g} \cdot \left(\frac{g_{\max}}{\|g\|_2}\right)$.

5.4 Adaptive Optimization Algorithms

- **RMSprop:** Maintains an exponentially decaying average of squared gradients:

$$\mathbf{v}_t = \alpha \mathbf{v}_{t-1} + (1 - \alpha) \mathbf{g}_t^2 \quad (8)$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\mathbf{v}_t} + \epsilon} \odot \mathbf{g}_t \quad (9)$$

where $\alpha \in [0.9, 0.999]$ and $\epsilon = 10^{-8}$.

- **Adam:** Computes biased first and second moment estimates with bias-correction:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2 \quad (10)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \quad (11)$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \hat{\mathbf{m}}_t \quad (12)$$

6 End-to-End Dry Run Trace (Concrete Walkthrough)

To verify the mathematical and operational mechanics of the engine, we perform a detailed trace of a single training step on an XOR configuration: Architecture $2 \rightarrow 2 \rightarrow 1$, using tanh hidden activation, σ output activation, MSE loss, and RMSprop optimization.

6.1 Initial State Setup

- **Sample Input:** $\mathbf{x} = [0.0, 1.0]^T$, **Target:** $y = 1.0$.
- **Hidden Layer Weights & Biases:** $\mathbf{W}^{[1]} = \begin{bmatrix} 0.5 & -0.2 \\ -0.4 & 0.8 \end{bmatrix}$, $\mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$.
- **Output Layer Weights & Bias:** $\mathbf{W}^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix}$, $b^{[2]} = 0.2$.
- **Hyperparameters:** Learning rate $\eta = 0.1$, RMSprop decay $\alpha = 0.9$, $\epsilon = 10^{-8}$, initial second moment $\mathbf{v}_0 = 0$.

6.2 Forward Pass Execution

1. **Hidden Net Inputs $\mathbf{z}^{[1]}$:**

$$\mathbf{z}^{[1]} = \begin{bmatrix} 0.5(0) - 0.2(1) + 0.1 \\ -0.4(0) + 0.8(1) - 0.1 \end{bmatrix} = \begin{bmatrix} -0.1000 \\ 0.7000 \end{bmatrix}$$

2. **Hidden Activations $\mathbf{a}^{[1]} = \tanh(\mathbf{z}^{[1]})$:**

$$\mathbf{a}^{[1]} = \begin{bmatrix} \tanh(-0.1000) \\ \tanh(0.7000) \end{bmatrix} = \begin{bmatrix} -0.0997 \\ 0.6044 \end{bmatrix}$$

3. Output Net Input $z^{[2]}$:

$$z^{[2]} = 0.7(-0.0997) - 0.5(0.6044) + 0.2 = -0.0698 - 0.3022 + 0.2 = -0.1720$$

4. Output Activation $a^{[2]} = \sigma(z^{[2]})$:

$$a^{[2]} = \frac{1}{1 + e^{0.1720}} = 0.4571$$

5. Loss Computation (MSE):

$$\mathcal{L} = \frac{1}{2}(a^{[2]} - y)^2 = \frac{1}{2}(0.4571 - 1.0)^2 = \frac{1}{2}(-0.5429)^2 = 0.1474$$

6.3 Backward Pass Execution

1. Output Error Delta $\delta^{[2]}$:

$$\begin{aligned}\sigma'(z^{[2]}) &= a^{[2]}(1 - a^{[2]}) = 0.4571(1 - 0.4571) = 0.2482 \\ \delta^{[2]} &= (a^{[2]} - y) \cdot \sigma'(z^{[2]}) = (-0.5429) \cdot 0.2482 = -0.1347\end{aligned}$$

2. Gradients for Output Layer:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[2]}} &= \delta^{[2]}(\mathbf{a}^{[1]})^T = -0.1347 \cdot [-0.0997, 0.6044] = [0.0134, -0.0814] \\ \frac{\partial \mathcal{L}}{\partial b^{[2]}} &= \delta^{[2]} = -0.1347\end{aligned}$$

3. Hidden Error Deltas $\delta^{[1]}$:

$$\begin{aligned}\tanh'(\mathbf{z}^{[1]}) &= 1 - (\mathbf{a}^{[1]})^2 = \begin{bmatrix} 1 - (-0.0997)^2 \\ 1 - (0.6044)^2 \end{bmatrix} = \begin{bmatrix} 0.9901 \\ 0.6347 \end{bmatrix} \\ \delta^{[1]} &= \left((\mathbf{W}^{[2]})^T \delta^{[2]} \right) \odot \tanh'(\mathbf{z}^{[1]}) = \begin{bmatrix} 0.7(-0.1347) \cdot 0.9901 \\ -0.5(-0.1347) \cdot 0.6347 \end{bmatrix} = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix}\end{aligned}$$

4. Gradients for Hidden Layer:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{[1]}} &= \delta^{[1]} \mathbf{x}^T = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix} [0.0, 1.0] = \begin{bmatrix} 0.0 & -0.0934 \\ 0.0 & 0.0428 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1]}} &= \delta^{[1]} = \begin{bmatrix} -0.0934 \\ 0.0428 \end{bmatrix}\end{aligned}$$

6.4 Parameter Update (RMSprop Step)

For output weight $W_{1,2}^{[2]}$ (current value -0.5 , gradient $g = -0.0814$):

$$\begin{aligned}v_1 &= 0.9(0) + 0.1(-0.0814)^2 = 0.0006626 \\ W_{1,2}^{[2]} &\leftarrow -0.5 - \frac{0.1}{\sqrt{0.0006626 + 10^{-8}}}(-0.0814) = -0.5 - \frac{0.1}{0.02574}(-0.0814) = -0.5 + 0.3162 = -0.1838\end{aligned}$$

Notice how the adaptive learning rate immediately applies a large corrective step ($+0.3162$) toward lowering the error, demonstrating the fast convergence characteristics of adaptive second-moment algorithms.

7 Interactive Workbench (GUI/UX) Architecture

7.1 Asynchronous Threading Model (LiveBridge)

To maintain an uncompromising 60 FPS user interface without dropped frames or stuttering during intensive training loops, MiniANN implements a thread-safe producer-consumer model using the `LiveBridge` abstraction:

- **Worker Thread (Producer):** Executes `Trainer::fit()`. At every epoch, it logs scalar metrics and snapshots network state to an internal ring buffer guarded by `std::mutex`.
- **UI Thread (Consumer):** A `QTimer` running at 33ms (30 FPS) reads from atomic counters (`g_bridge.epoch`) and pulls batch metrics, refreshing canvas widgets without blocking.
- **Responsive Interruption:** The GUI thread communicates cancellation via atomic boolean flags (`stopRequested`), halting the background worker in sub-millisecond time.

7.2 Visualization Canvas Modules

- **Loss & Accuracy Telemetry:** Live line charts displaying training and validation metrics simultaneously, featuring auto-scaling axes, moving averages, and best-epoch markers.
- **2D Decision Boundary Heatmap:** Computes a dynamic 60×60 dense grid inference over feature space (x_1, x_2) , rendering classification contours alongside colored data points.
- **Feedforward Network Graph:** Interactive SVG/raster rendering of network layers. Node sizes indicate activation strength, while connection thickness and color denote weight polarity and magnitude.
- **Confusion Matrix:** Real-time multiclass performance grid showing true positives, false positives, precision, recall, and F1-score.

7.3 UI Polish & Workflow Enhancements

- **Structured Partition Card:** Clean 3-column input card for Train/Val/Test splits with auto-clamping, stacked color split bar, and partition legends.
- **Dataset Metadata HUD:** High-contrast HTML metadata panel displaying dimensions, class counts, and sample counts.
- **Keyboard Navigation:** Quick keyboard shortcuts (`Space` / `F5` to start/stop, `Esc` to abort, `Ctrl+R` to reset defaults, `Ctrl+E` to export report, `Ctrl+S`/`Ctrl+O` for model I/O).

8 Automated Verification & Test Suite

MiniANN MVP maintains an exhaustive test harness consisting of six automated test binaries compiled via `build.ps1`:

9 Conclusion & Future Roadmap

The MiniANN MVP project successfully achieves its pedagogical and architectural objectives. By enforcing a zero-external-library constraint, the framework validates that clean, robust, and extensible deep learning software can be engineered directly in standard C++20 using disciplined object-oriented design principles.

Test Executable	Scope & Coverage	Pass Criteria
<code>gradient_check.exe</code>	Analytic backprop vs. finite difference approximations	Global relative error $\frac{\ g_{\text{analytic}} - g_{\text{numerical}}\ }{\ g_{\text{analytic}}\ + \ g_{\text{numerical}}\ } < 10^{-7}$
<code>test_correctness.exe</code>	XOR convergence and mathematical sanity	100% classification accuracy on XOR, AND, OR benchmarks
<code>test_training.exe</code>	Multi-optimizer descent & regularization	Verified monotonic loss reduction for SGD, Momentum, Adam, RMSprop; gradient clipping validation
<code>test_callbacks.exe</code>	Observer pattern & early stopping	Verified patience, min-delta, NaN abort, and callback aggregation
<code>test_patterns.exe</code>	Verification of 9 GoF design patterns	Validates composite validators, visitors, factories, schedulers, and strategy independence
<code>test_choices.exe</code>	Activations, losses, and factory tokens	Analytic derivatives for all 8 activations (including ELU, Swish); cross-entropy gradient checks

Table 2: MiniANN Automated Verification and Test Harness Overview

9.1 Architectural Readiness for 3D Visualization

Due to the strict encapsulation of the `LiveBridge` telemetry interface, future enhancements—such as the planned **3D OpenGL / Vulkan real-time activation visualizer**—can be integrated as independent consumers. The 3D render pipeline can ingest existing `VisualizationState` snapshots without requiring a single modification to the core neural network backend or the training loop, fully demonstrating the extensibility benefits of the Open-Closed Principle (OCP).